The TaaS Company

INFRASTRUCTURE & CODE ASSESSMENT

Blue Lagoon
Magento 2 Production Audit

A consolidated technical assessment of the Magento Open Source 2.4.8-p4 production deployment serving **www.blue-lagoon.nl** (B2C) and **b2b.blue-lagoon.nl** (B2B) — covering custom code quality, the production Docker stack, caching topology, and a prioritised implementation roadmap.

PLATFORM
Magento OS 2.4.8-p4

RUNTIME
PHP 8.3 · MySQL 8 · Redis 7

PREPARED BY
The TaaS Company

REPORT DATE
2 June 2026

Contents

Scope of this report

This assessment is scoped to the **production environment** only. It consolidates four working analyses (infrastructure & code audit, setup analysis, production Redis analysis, and the Redis deep-dive) into a single deliverable, removing duplication and presenting one prioritised set of recommendations. Severity ratings follow the convention: CRITICAL HIGH MEDIUM LOW INFO.

1. Executive summary

Blue Lagoon runs a mature Magento Open Source 2.4.8-p4 storefront on PHP 8.3, fronted by Varnish 7.5 and backed by MySQL 8 and Redis 7, with an external OpenSearch cluster at `106.201.236.21:9800`. The platform carries a substantial custom suite — **26 modules across six vendor namespaces** — with the **Typify_Ofbiz** ERP integration providing B2B pricing, stock, order and returns sync.

The codebase is functional and in active production, but two custom plugins on Magento hot paths dominate the performance profile, and the production Docker stack is missing several standard build, hardening and tuning steps. The good news: every top finding is well-understood and fixable without re-platforming, and the highest-impact items can be addressed within the first week.

26	1004	22+	0
CUSTOM MODULES	COMPOSER PACKAGES	OBJECTMANAGER MISUSES	SERVICE HEALTHCHECKS

The five issues that matter most

#	Finding	Severity	Why it matters
1	**Synchronous cURL per SKU** in `Typify\Ofbiz\Plugin\Product::afterGetPrice`	CRITICAL	B2B price lookups call `ofbiz.typify.com` one SKU at a time with **no timeout**. A 60-product category page can fire 60 sequential HTTP calls on first view; one slow ERP response stalls the whole PHP-FPM worker.
2	**FPC identifier plugin** uses `ObjectManager::getInstance()` on every cacheable request	CRITICAL	`Typify\Ofbiz\Plugin\App\PageCache\Identifier` resolves dependencies via the ObjectManager inside the full-page-cache key generator — a path Magento keeps deliberately minimal — affecting B2C traffic too.
3	**Production image bakes neither** `di:compile` **nor** `static-content:deploy`	HIGH	Containers boot slow and generate DI/assets at runtime. Two conflicting PHP-FPM pool files also ship together, leaving effective concurrency undefined.
4	**Single shared Redis** for cache + FPC tags + sessions with `allkeys-lru`	HIGH	Under cache pressure, Redis can evict session keys, causing random customer logouts. Persistence is off, so a restart drops every session.
5	**No service tuning or resilience baseline**	HIGH	MySQL ships with default InnoDB settings, nginx runs `error_log debug` with `worker_processes 1`, the cron is a bare shell loop, and no service defines a healthcheck.

Overall posture

Area	Verdict	Severity
Magento version & PHP runtime	Current and supported — 2.4.8-p4 on PHP 8.3	OK
Custom modules (Typify / Ethnic)	Mixed quality; ObjectManager antipattern, per-SKU HTTP, FPC plugin on every request	HIGH

Production Docker stack	No in-image compile/deploy, conflicting FPM pools, no healthchecks, untuned MySQL	HIGH
Caching (Redis + Varnish)	Sound 2-level + Varnish design, but single Redis shares cache, FPC tags and sessions	MEDIUM
Indexers / message queue	RabbitMQ absent from prod compose; async work falls back to DB queue	MEDIUM
Search	External OpenSearch on `106.201.236.21:9800` — logs show a `NoNodesAvailable` outage on 2026-05-18; single node, no failover or alerting	HIGH
Inventory (MSI)	All MSI modules disabled — intentional single-source performance win	INFO

2. System & inventory overview

2.1 Production topology

```
Internet / Cloudflare
   → Varnish 7.5  (:6081, 2 GB malloc, ESI on)   # HTML full-page cache
        → Nginx  (:80, FastCGI)
             → PHP-FPM 8.3  (:9000)
                  → MySQL 8.0.41   (db service)
                  → Redis 7         (DB0 cache · DB1 FPC tags · DB2 sessions)
                  → OpenSearch       (external: 106.201.236.21:9800, prefix b2c_prod)

Containers: php · nginx · varnish · cron · db · redis
Not in compose: search cluster (external), RabbitMQ, queue consumers,
                reverse-proxy/TLS terminator, log shipper, backup job
```

2.2 Magento core & key third-party packages

Package	Version	Purpose / note
`magento/product-community-edition`	2.4.8-p4	Current. One active patch reverts the 2.4.8-p3 CSP `SubresourceIntegrity` refactor — re-evaluate at every upgrade.
`paynl/magento2-plugin`	3.10.0	Pay.nl — primary EU/NL payment path
`paypal/module-braintree`	4.7.0	Enabled but **misconfigured** — log shows `merchantId needs to be set`
`mailchimp/mc-magento2`	103.4.57	MailChimp sync — active
`wyomind/simplegoogleshopping`	17.0.9	Google Shopping feed (private repo)
`boldcommerce/magento2-ordercomments`	1.9.0	Extended by `Typify_BoldOrderCommentExtend`
`experius/module-postcode`	^1.2	NL/BE postcode lookup
`mageplaza/module-smtp`	^4.7	SMTP gateway
`magento/inventory-metapackage`	—	All MSI modules disabled in `config.php`

Total: 1004 Composer packages in `composer.lock`. Note: `composer/composer` is pinned to `@alpha` — replace with an explicit semver for reproducible builds.

2.3 Custom modules (`src/app/code`)

26 modules in six vendor namespaces; `src/app/code` totals 63 MB, of which `Typify_TerrariumConfigurator` alone is 53 MB.

#	Module	Size	Status	Function
1	`Typify_TerrariumConfigurator`	53M	enabled	Visual aquarium configurator — 588 PNGs in `view/frontend/`

2	`Phpro_CookieConsent`	3.0M	enabled	EU cookie consent
3	`Typify_Projects`	1.5M	enabled	CMS "Projects" entity
4	`Typify_Faq`	1.0M	enabled	FAQ + opening hours
5	`Typify_Blog`	492K	enabled	Blog / articles
6	`Typify_Caresheets`	468K	enabled	Animal / plant care sheets
7	`Typify_TypifyWidgets`	416K	enabled	Storefront widgets, category overrides
8	`Mageplaza_GoogleRecaptcha`	396K	enabled	reCAPTCHA — **duplicates** native `Magento_ReCaptcha*`
9	`Typify_Ofbiz`	384K	enabled	**Ofbiz ERP integration (B2B core)**
10	`Meetanshi_ImageClean`	364K	enabled	Unused-media cleanup (cron + CLI)
11	`Typify_Checkout`	356K	enabled	Checkout customisations (date, fair, pickup)
12	`Typify_Shelter`	328K	enabled	"Shelter" / reservation flow
13	`Typify_Returns`	276K	enabled	Returns portal → Ofbiz export
14	`Typify_Calendar`	260K	enabled	Calendar / events + REST API
15	`Typify_Assembly`	252K	enabled	Assembly reservations
16	`Typify_Vacancies`	240K	enabled	Job vacancies
17	`Meetanshi_Extensions`	148K	enabled	Advertising / upsell admin block
18	`Tawk_Widget`	120K	enabled	Tawk.to live chat
19	`Ethnic_EanPriceUpload`	108K	enabled	Admin EAN/price CSV uploader
20	`Typify_CustomFilters`	96K	**disabled**	Multi-select layered navigation — still on disk
21–26	`Typify_BestsellerWidget`, `BoldOrderCommentExtend`, `AdaptiveResize`, `Usps`, `Ethnic_ConfigurableOptionSorting`, `Ethnic_UnifiedCache`	small	enabled*	Widgets, image resize, USP block, option sorting, multi-store cache flush observer

* `Ethnic_UnifiedCache` is enabled in `src/app/etc/config.php` but **not present in `prod-conf/config.php`** — the admin "Flush Cache" → edge purge observer is therefore inactive in production (see §5.4).

2.4 Theme & disabled modules

The storefront theme is `Typify/BlueLagoon` (parent `Magento/blank`), titled "Blue Lagoon 2018". It overrides 14 core modules plus `Typify_TypifyWidgets`; every Luma template has been re-implemented locally, which materially increases `setup:static-content:deploy` time. In `config.php`, the entire `Magento_Inventory*` MSI stack (~50 modules) is intentionally disabled for the single-warehouse model, and `Typify_CustomFilters` is disabled but still installed on disk.

3. Code-quality audit

3.1 CRITICAL Synchronous external HTTP per product

File: `Typify/Ofbiz/Plugin/Product.php` — `afterGetPrice()`. For B2B customers this plugin replaces each product's price with a value fetched live from `https://ofbiz.typify.com/php-sync/product/price/{sku}`, cached only per-session and built one SKU at a time.

```
public function afterGetPrice($subject, $result) {
    $isB2BStore = $this->_dataHelper->getCatalog() == 'B2B';
    $customerId = $this->customerSession->getCustomer()->getData('ofbiz_id');
    if ($result && $customerId && $isB2BStore) {
        return $this->getPriceForCustomer($customerId, $subject->getSku(), $result) ?: $result;
    }
    return $result;
}
// curl_init/curl_exec with NO CURLOPT_TIMEOUT and NO CURLOPT_CONNECTTIMEOUT
```

Issue	Consequence
No connect/total timeout (default 0 = infinite)	One slow Ofbiz response stalls the entire PHP-FPM worker
No batch endpoint — N products = N round trips	A 60-item category page issues 60 sequential HTTP calls on first session view
Runs on `Product::getPrice`, called many times per page	Multiplied across block rendering, layered nav, mini-cart, related products
Session-only cache, evaporates on logout	First page view of every session is the slowest; never pre-warmed
Bypasses Magento's price index	B2B customers get on-the-fly Ofbiz prices even where a catalog rule applies

Recommended fix

Add `CURLOPT_CONNECTTIMEOUT_MS=500` and `CURLOPT_TIMEOUT_MS=1500`, bailing to the default price on timeout. Move the cache to server-side Redis keyed by `customer_id + sku` (TTL ≤ 5 min). Add a `Collection::afterLoad` plugin that pre-fetches all visible SKUs for the active B2B customer in **one** batched request. Long term, sync Ofbiz prices into Magento's customer-group-price tables on a schedule and treat the plugin as a read-through fallback only.

3.2 CRITICAL FPC identifier plugin on every cacheable request

File: `Typify/Ofbiz/Plugin/App/PageCache/Identifier.php`. Adds the B2B `customerId` to the full-page-cache hash key, but resolves its dependencies through the ObjectManager on a path Magento keeps deliberately minimal.

```
public function getB2bCustomerId() {
    $objectManager = ObjectManager::getInstance();        // hot path
    $dataHelper    = $objectManager->get(Data::class);
    $isB2BStore    = $dataHelper->getCatalog() == 'B2B';
    // ...
}
```

This runs on **every** cacheable request, including B2C, so the ObjectManager lookup is pure overhead for the majority of traffic. The dependencies are invisible to static analysis and bypass the compiled DI graph, defeating tracing and interception. A related defect: the source references `Data` and `Session` without `use` imports, which can fatally error on B2B stores once that branch is reached.

> **Recommended fix**
> Constructor-inject `Typify\Ofbiz\Helper\Data` and `Magento\Customer\Model\Session`, add the missing `use` statements, and short-circuit `getB2bCustomerId()` before any session access on B2C.

3.3 MEDIUM ObjectManager antipattern — 22+ occurrences

Direct `ObjectManager::getInstance()` usage appears in 22+ source files across Typify and Ethnic modules, defeating Magento's interception and compilation guarantees. Representative cluster:

```
Typify/Ofbiz/Plugin/App/PageCache/Identifier.php
Typify/TerrariumConfigurator/{Model/Terrarium, Lib/PriceCalculator, Lib/Field, Lib/Settings ×2}.php
Typify/Calendar/{Helper/Data ×2, Model/Event, Model/EventManagement}.php
Typify/Assembly/{Model/Reservation ×2, Block/Cart, Block/View ×2, Block/Sidebar}.php
Typify/Projects/{Controller/Adminhtml/Project, Model/Project ×2, Helper/Data ×2}.php
Typify/Faq/Model/DatepickerSettings.php · Typify/BestsellerWidget/Block/Widget/...php
Meetanshi/{Extensions/Block/.../Extensions, ImageClean/Observer/ProductDelete}.php
Typify/CustomFilters/* (module disabled)
```

Migrate these to constructor injection during a maintenance window; combined with `setup:di:compile` this yields measurable autoload and interception speedups.

3.4 MEDIUM Other code smells

Finding	Location	Risk
`<sequence>` placed outside the `<module>` element	`Typify/Shelter/etc/module.xml`	Magento ignores it; load order undefined
Two reCAPTCHA stacks active simultaneously	`Mageplaza_GoogleRecaptcha` + 29 native `Magento_ReCaptcha*`	Double validation, possible double-prompt on admin login
Disabled module still installed	`Typify/CustomFilters/`	Dead code, lingering plugin on `CatalogSearch`
Same observer registered twice	`Ethnic_UnifiedCache/etc/adminhtml/events.xml`	Edge flush runs twice per admin click
`set_time_limit(0)` in cron	`Typify/Ofbiz/Cron/Import.php`	Hung crons run forever; needs a lockfile
Writes outside Magento `Filesystem` allow-list	`Typify_AdaptiveResize` → `media/resizes/cache/`	Logged error on every page render
Full cache flush inside product import	`Typify/Ofbiz/Cron/ImportProduct.php`	`cacheManager->clean(getAvailableTypes())` every 30 min → sitewide cold start
Leftover test route + empty `execute()`	`Ofbiz/Observer/RestrictWebsite`, `Vacancies/.../Rewrite`	Configurator route reachable on B2B; dead admin action

3.5 Custom cron schedule & collisions

Job	Schedule	Note
`meetanshi_imageclean_cron`	`* * * * *`	**Every minute** — 1440 runs/day for a maintenance task
`ofbiz_import_product`	`*/30 * * * *`	Heavy; triggers full cache flush path
`ofbiz_import_stock`	`30 * * * *`	Hourly
`ofbiz_import_order / _status`	`40 / 45 * * * *`	Hourly
`ofbiz_import_return`	`15 * * * *`	**Collides** with customers job — both hit Ofbiz API
`ofbiz_import_customers`	`15 * * * *`	**Collides** with return job at `*:15`

Stagger the two `*:15` Ofbiz jobs and reconsider the per-minute ImageClean schedule.

3.6 Active runtime errors (from production `var/log`)

Observed in the live logs under `var/log` (`exception.log`, `system.log`, `typify-debug.log`, cross-referenced with `pay.log`):

Observed error	Root cause & action
CRITICAL `OpenSearch ... NoNodesAvailableException: No alive nodes found in your cluster` — repeated on 2026-05-18, hitting category, search and search-suggestion rendering (`exception.log`)	The external OpenSearch node (`106.201.236.21:9800`, index `b2c_prod`) became unreachable and storefront category/search pages threw fatals. It is a single external node with no failover, retry budget, or health alerting. Add connection retry + timeout, alert on cluster health, and evaluate a second node / managed HA (see §4.9).
CRITICAL `InvalidArgumentException: 'value' must be type of string, NULL given` and `nl2br(): Passing null ... is deprecated` — via `Typify/Calendar/Block/View.php:48` and `view.phtml:30` (`exception.log`)	`Typify_Calendar` passes `NULL` CMS content into the widget filter and `nl2br()` when an event has no body. Null-guard `getCmsFilterContent()` and the template (default to an empty string) to stop the fatal on calendar event pages.
MEDIUM GraphQL `'zoeken' of 'filter' invoerargument is vereist` and `Field "visibility" is not defined by type "ProductAttributeFilterInput"` — recurring (`exception.log`)	A repeat caller hits the `products` GraphQL endpoint with no required `search` / `filter` argument, and elsewhere with an unsupported `visibility` filter. Identify the client (feed, monitor, or headless integration) and fix the query, or expose `visibility` as a filterable attribute. Adds steady log noise on every hit.
MEDIUM `QuoteItemId not found in QuoteItems` — `Typify/Shelter/Observer/SalesModelServiceQuoteSubmitBeforeObserver.php` (`typify-debug.log`)	Fires during real order submission — timestamps line up with Pay.nl orders in `pay.log` (e.g. 2026-05-18 11:52:08 ↔ order 23924). The Shelter submit observer looks up a quote item that is no longer present. Harden the lookup and confirm Shelter cart items survive to `quote_submit_before`.
MEDIUM `Braintree\Configuration::merchantId needs to be set` (`system.log`)	Braintree enabled with empty/wrong-scope credentials while Pay.nl is the live gateway. Configure or disable Braintree to stop the error and rule out a silently broken method.

Logging hygiene: `pay.log` emits `PAY.DEBUG` entries (including transaction identifiers) in production. Consider raising the Pay.nl log level to `NOTICE` outside active debugging to cut noise and avoid persisting transaction references.

4. Production Docker stack (`docker-prod/`)

4.1 What's strong today

> **Production strengths worth preserving**
> The PHP container drops all capabilities (`cap_drop: ALL`) and sets `no-new-privileges:true`. Network isolation is good — only Varnish exposes a host port, bound to `127.0.0.1:6081`. The Varnish VCL is clean: ESI, GraphQL cache-id support, grace handling for sick backends, and marketing query-string stripping (`utm_*`, `gclid`, `fbclid`, `mc_*`, etc.). Nginx denies the sensitive paths (`/app`, `/bin`, `/setup`, `/vendor`, `composer.*`, `auth.json`, `cron.php`). OPcache uses `validate_timestamps=0`, correct for production.

4.2 PHP image build — HIGH

#	Issue	Severity	Fix
P1	`composer require wyomind/...` runs *inside* the Dockerfile, mutating `composer.json` / `.lock` at build time	HIGH	Add Wyomind to root `composer.json`; the lockfile is the single source of truth
P2	No `setup:di:compile` baked in	HIGH	Run `bin/magento setup:di:compile` after `composer install`
P3	No `setup:static-content:deploy` baked in — `pub/static` empty on cold boot	HIGH	`setup:static-content:deploy -f nl_NL en_US -j 4`
P4	`composer install` includes dev dependencies	MEDIUM	Add `--no-dev --optimize-autoloader --classmap-authoritative`
P5	Single-stage build ships the full toolchain in the runtime image	LOW	Adopt a builder stage; copy `vendor/` + compiled output to a slim runtime

4.3 PHP-FPM pools — conflict

Two pool files ship in the image and only one wins at runtime: a minimal `php-fpm.conf` pool (`pm.max_children = 10`) and a `www.conf` pool (`pm.max_children = 20`, `memory_limit = 4096M`, `max_execution_time = 18000`). Effective behaviour is undefined until verified with `php-fpm -tt`.

Setting	Current	Recommendation
`pm.max_children`	≤ 20	Right-size from `(usable RAM ÷ per-request memory)` — target 32–48 on an 8-vCPU / 32 GiB box
`memory_limit` per worker	4096M	768M–1024M storefront; raise per-endpoint for imports via `fastcgi_param PHP_VALUE`
`max_execution_time`	18000s (5 h)	Scope long limits to import/admin paths only, not storefront PHP
`pm.max_requests`	unset	Set ~500 to bound worker memory growth

4.4 OPcache & JIT

`opcache.max_accelerated_files = 60000` is likely too low — Magento with 26 custom modules plus theme typically loads 80k–130k files. Raise to `130000` and verify with `opcache_get_status()`. Because

`validate_timestamps=0` is set, a code change without a container rebuild serves stale bytecode — document the deploy step (rebuild image, or `kill -USR2 1` on the php container). After benchmarking, consider tracing JIT (`opcache.jit=tracing`, `opcache.jit_buffer_size=256M`) for a modest 5–15% gain on CPU-bound paths.

4.5 Nginx — MEDIUM

Setting	Current	Fix
`error_log`	`debug`	`warn` — debug in production creates huge log volume and measurable cost
`worker_processes`	`1`	`auto` for multi-vCPU hosts
`gzip`	off	Enable for `text/*`, `application/json`, `application/javascript`, `application/xml`
`server_name`	`b2cnewserver.ethnicinfotech.in`	`www.blue-lagoon.nl` + `b2b.blue-lagoon.nl`, one server block per host
`fastcgi_buffers`	`16 16k`	`64 32k` + `fastcgi_buffer_size 64k` for long `X-Magento-Tags` headers

4.6 MySQL — HIGH

The `my.cnf` sets only `log-bin-trust-function-creators=1` — no InnoDB tuning at all, which is inappropriate for production e-commerce. The auth plugin is `mysql_native_password` (deprecated in 8.4, removed in 9.x). Recommended baseline:

```
[mysqld]
innodb_buffer_pool_size        = 4G      # ~70% of DB container RAM
innodb_buffer_pool_instances   = 4
innodb_log_file_size           = 1G
innodb_flush_log_at_trx_commit = 2
innodb_flush_method            = O_DIRECT
innodb_io_capacity             = 2000
innodb_io_capacity_max         = 4000
max_connections                = 400
table_open_cache               = 8000
thread_cache_size              = 100
tmp_table_size                 = 256M
max_heap_table_size            = 256M
slow_query_log                 = 1
long_query_time                = 1
```

Migrate to `caching_sha2_password` (supported by Magento ≥ 2.4.5) on a planned window.

4.7 Cron — non-resilient shell loop HIGH

```
cron:
  command: sh -c "while true; do
      su -s /bin/sh www-data -c 'cd /app && php bin/magento cron:run'
      sleep 60
    done"
```

No logging, no failure backoff, no overlap protection — a hung `cron:run` accumulates stuck PHP processes in the same container, and the existing `.docker/php/8.3/magento-cron` crontab file is unused. Replace with `cron -f` as PID 1 using that crontab file, or wrap with `supervisord` for autorestart and log rotation.

4.8 What's missing from compose

Gap	Impact
No healthchecks on any service	Compose can't sequence MySQL/Redis readiness before PHP/cron start
No queue consumers / RabbitMQ	Indexers run "Update by Schedule" and need consumers; async/bulk work falls back to the slower DB queue. Confirm `env.php` does not declare `queue.amqp` against an absent host.
No log shipper, no backup job	Logs land only in named volumes; no scheduled `mysqldump`
Volume name mismatch	`magento-prod_magento_media` (external) is declared but unused; services mount the non-external `magento_media`. Pick one canonical volume.
No profiling variant	Production profiling needs a separate image with `php-spx` or Blackfire baked in

4.9 Search wiring

Production uses the Magento `opensearch` engine against the external host `106.201.236.21:9800`, index prefix `b2c_prod`, with a 15 s server timeout. Ensure the container `entrypoint.sh` does not reset the port to `9200` on restart, and reindex `catalogsearch_fulltext` after any change.

> **Search is a single point of failure**
> The production `exception.log` recorded a `NoNodesAvailableException` on 2026-05-18 where the single external node was unreachable and storefront category, search and suggestion pages threw fatal errors. Treat search as a critical dependency: add a connection retry budget and tight timeouts, alert on `_cluster/health`, and evaluate a second node or managed HA so node loss degrades gracefully instead of fataling the page.

Health check:

```
curl -s "http://106.201.236.21:9800/_cluster/health?pretty"
bin/magento config:show catalog/search/engine
bin/magento indexer:reindex catalogsearch_fulltext
```

5. Redis & caching topology

Production uses a **single Redis 7 instance** for three roles, with Varnish — not Redis — serving the rendered HTML full-page cache. The pain points here are rarely a hard "Redis is down" failure; they are a stack of configuration and code interactions that surface as random logouts, slowdowns every 30 minutes, and purge errors in the logs.

Redis DB	Role	Backend / notes
0	Default application cache (config, layout, block_html, EAV…)	`Cm_Cache_Backend_Redis`, prefix `mgt_`, gzip ≥ 20 KB
1	Page-cache backend (tags / invalidation metadata)	Same instance, `compress_data=0`. Often near-empty by design — Varnish holds the HTML.
2	Customer / admin sessions	`max_concurrency=24`, locking on, gzip ≥ 2 KB

5.1 The three highest Redis risks

Severity	Risk	Effect
HIGH	Shared 3 GB instance with `allkeys-lru`	Under cache pressure Redis can evict `sess_*` keys in DB 2 → random customer logouts and empty carts
HIGH	Persistence disabled (`--save '' --appendonly no`)	A container restart drops all cache *and* all sessions — every customer logged out
MEDIUM	Ofbiz product import flushes all cache types every 30 min	`cacheManager->clean(getAvailableTypes())` causes a Redis delete storm + sitewide cold start every half hour

5.2 Redis service definition (current)

```
redis:
  image: redis:7-alpine
  command: redis-server --requirepass ${REDIS_PASSWORD}
    --maxmemory 3gb --maxmemory-policy allkeys-lru
    --maxmemory-samples 10 --save '' --appendonly no
  mem_limit: 3500m       # no healthcheck; no depends_on from cron
```

The `cron` container has no `depends_on: redis`, so on a cold start it can run imports before Redis accepts connections — producing transient errors in `var/log`.

5.3 Recommended target topology

Split cache and sessions into two failure domains so cache pressure can never evict a session, and a cache restart can never log a customer out:

`redis-cache`	`redis-session`

`--maxmemory 2gb`
`--maxmemory-policy allkeys-lru`
DB0 cache · DB1 FPC tags
persistence off (rebuildable)
	`--maxmemory 512mb`
	`--maxmemory-policy noeviction`
	DB0 sessions only
	`--appendonly yes --appendfsync everysec`

Then point `REDIS_HOST=redis-cache` and `SESSION_REDIS_HOST=redis-session`. Also raise `connect_retries` to 3 and `read_timeout` to ~30 for cron-heavy windows, and enable a Redis-backed lock provider (`'lock' => ['provider' => 'redis']`) to take lock traffic off MySQL.

5.4 Edge purge & `env.php` generation

Two operational notes carry real risk. First, `Ethnic_UnifiedCache` (which fans an admin "Flush Cache" out to Cloudflare + Varnish) is **not enabled in `prod-conf/config.php`**, so admin flushes do not propagate to the edge in production — either enable it or document that edge purges are handled by CLI. Second, confirm the FPC purge target resolves to the `varnish` service name (not a stale hostname or `localhost`) so Magento's Varnish invalidation succeeds — this is a `core_config_data` setting, independent of Redis health:

```
SELECT path, value FROM core_config_data
WHERE path LIKE 'system/full_page_cache%';
-- backend_host should be: varnish   (not localhost / public domain)
```

> **Secrets hygiene**
> `prod-conf/env.php` contains live secrets and the deploy path embeds a crypt key in shell. Generate `env.php` once at deploy time and mount it read-only (or inject from a secrets manager) rather than regenerating it on every container start, which silently reverts manual fixes and risks password/host drift between `.env` and Redis.

6. Heavy plugin / observer map

The interception points where custom code touches Magento hot paths. The first two are the performance-critical ones covered in §3.

```
Magento\Framework\App\PageCache\Identifier::getValue
  └─ plugin: Typify\Ofbiz\Plugin\App\PageCache\Identifier       # FPC hot path — every request

Magento\Catalog\Model\Product::getPrice
  └─ plugin: Typify\Ofbiz\Plugin\Product::afterGetPrice         # cURL to ofbiz.typify.com

Magento\Customer\Model\Registration::isAllowed
  └─ plugin: Typify\Ofbiz\Plugin\Customer\...\DisableRegistration

Magento\Checkout\Block\Checkout\LayoutProcessor::process
  └─ plugin: Typify\Checkout\Plugin\Checkout\LayoutProcessorPlugin

Magento\Checkout\Model\ShippingInformationManagement::saveAddressInformation
  └─ plugin: Typify\Checkout\Plugin\Quote\SaveToQuote

Magento\ConfigurableProduct\...\Product\Type\Configurable
  └─ plugin: Ethnic\ConfigurableOptionSorting\...\Configurable    # usort on option load

Magento\Framework\View\Result\Page::renderResult
  └─ plugin: Typify\Ofbiz\Plugin\Result\Page

Magento\Catalog\Block\Product\View
  └─ plugin: Typify\TypifyWidgets\Plugin\Catalog\Block\Product\View

Magento\CatalogSearch\Model\Adapter\Mysql\Filter\Preprocessor::process
  └─ plugin: Typify\CustomFilters\Plugin\...                      # module DISABLED

Events
  controller_action_predispatch       Mageplaza_GoogleRecaptcha ·
Typify\Ofbiz\Observer\RestrictWebsite · Meetanshi_Extensions
  admin_user_authenticate_before      Mageplaza_GoogleRecaptcha
  adminhtml_cache_flush_system|all     Ethnic_UnifiedCache (duplicated; disabled in prod-conf)
  catalog_product_delete_after_done    Meetanshi_ImageClean
  checkout_cart_product_add_after      Typify_Shelter
  checkout_submit_before               Typify_Checkout
  sales_model_service_quote_submit_*   Typify_Shelter · Typify_Checkout
  payment_method_is_active             Typify_Checkout (banktransfer)
  customer_login                       Typify_Ofbiz
  layout_load_before                   Typify_Ofbiz (body class) — frontend + admin
```

Module dependency map

```
Typify_Checkout              ─┬─ Magento_Checkout
                              └─ Typify_Calendar
Typify_Assembly              ─── Typify_Shelter   # invalid <sequence> placement
Typify_Shelter               ─── Magento_Checkout
Typify_BoldOrderCommentExtend ── Bold_OrderComment
Typify_CustomFilters         ─┬─ Magento_CatalogSearch   (DISABLED)
                              └─ Magento_Swatches
```

```
Ethnic_ConfigurableOptionSorting — Magento_ConfigurableProduct
Mageplaza_GoogleRecaptcha   ——— Mageplaza_Core
```

No circular dependencies detected. The main latent risk is silent ordering — several Typify modules omit a valid `<sequence>` even where they patch core models. Address during the next major upgrade.

7. Validation methodology

Each fix should be measured, not assumed. Validate against a production-equivalent instance built from the same Dockerfile plus a `php-spx` profiling variant.

7.1 Profiling targets

Drive the profiler at the three pages where custom code concentrates, then read the flamegraph for time spent in the interception points below:

- Category listing with > 50 products (exercises the Ofbiz price plugin)
- Configurable PDP (exercises `Ethnic_ConfigurableOptionSorting` + the configurator)
- Checkout (exercises `Typify_Checkout` + `Typify_Calendar`)

```
Look for time in:
  PageCache\Identifier\Interceptor::getValue   # Ofbiz FPC plugin
  Magento\Catalog\Model\Product::getPrice      # Ofbiz cURL
  View\Asset\Merged::initialize                # mixins.js error chain
  Cache\Backend\Redis::save / ::load           # compression cost
```

7.2 Load test & live counters

```
wrk -t8 -c200 -d60s --latency https://<host>/<category-url>
wrk -t8 -c200 -d60s --latency https://<host>/<configurable-pdp>
```

Layer	What to watch
Varnish	`varnishstat` → `MAIN.cache_hit` / `cache_miss` / `ws_*` overflow
Redis	`INFO stats` → `keyspace_hits/misses`, **`evicted_keys`** (treat > 0 in business hours as a session-eviction incident), `blocked_clients`
MySQL	`SHOW ENGINE INNODB STATUS`, `SHOW PROCESSLIST`, slow-query log
PHP-FPM	status page → `accepted conn`, `listen queue`, **`max children reached`**

7.3 Static analysis

Run `phpstan` and `phpcs` (Magento coding standard) over `app/code/Typify` and `app/code/Ethnic`. Expect warnings on every `ObjectManager::getInstance()` call site and missing type hints — use the count as a burn-down metric for §3.3.

8. Implementation & fix roadmap

Work is sequenced into four phases. Each phase is independently shippable and ordered so that the highest-impact, lowest-risk fixes land first. Effort is a rough engineering estimate; **Validate** names the signal that proves the fix worked.

Phase 0 — Stabilise WEEK 1

Config and code changes that remove the sharpest failure modes. No architectural change; deployable behind the existing image rebuild.

ID	Action	Effort	Validate
S0-1	Add cURL connect/total timeouts to `Ofbiz\Plugin\Product` and every other curl caller (`UnifiedCache\VarnishPurger` , `Returns\Ajax\Index` , `Ofbiz\Cron\Import`)	0.5 d	No worker stalls when Ofbiz is slow; p95 PDP latency stable
S0-2	Refactor `Ofbiz\Plugin\App\PageCache\Identifier` to constructor DI + missing `use` imports; short-circuit on B2C	0.5 d	SPX shows identifier off the hot path; no fatal on B2B
S0-3	Bake `setup:di:compile` + `setup:static-content:deploy -f` into the image; add `--no-dev --optimize-autoloader`	1 d	Cold-boot serves compiled code; `mixins.js` error gone
S0-4	Consolidate PHP-FPM pools to one file; `memory_limit` → 1024M; remove the duplicate pool	0.5 d	`php-fpm -tt` shows a single deterministic pool
S0-5	Nginx: `error_log warn` , `worker_processes auto` , enable gzip, set real `server_name`	0.5 d	Log volume drops; gzip on HTML/JS/CSS confirmed
S0-6	Raise `opcache.max_accelerated_files` to 130000	0.25 d	`opcache_get_status()` shows no cached-keys saturation
S0-7	Verify FPC purge host resolves to `backend_host=varnish` ; configure or disable Braintree	0.5 d	Varnish invalidation succeeds on product save; Braintree errors gone from log
S0-9	Null-guard `Typify_Calendar` (`getCmsFilterContent()` + `view.phtml` `nl2br`) against empty event content	0.25 d	No `InvalidArgumentException` / `nl2br` fatals on calendar event pages
S0-8	Remove the full-cache-flush call from `Ofbiz\Cron\ImportProduct` ; use scoped tag invalidation	0.5 d	No sitewide cold start on the 30-min import; `evicted_keys` flat

Phase 1 — Resilience & tuning WEEKS 2–4

Make the stack survive restarts and load, and split shared resources into independent failure domains.

ID	Action	Effort	Validate
S1-1	Split Redis into `redis-cache` (lru, no persistence) and `redis-session` (noeviction, AOF); repoint env vars	1 d	Cache pressure no longer evicts sessions; restart keeps users logged in
S1-2	Add healthchecks to all services + `depends_on` with conditions; add `depends_on: redis` on cron	1 d	PHP/cron never start before DB/Redis ready; no cold-start Redis errors
S1-3	Replace the cron shell loop with `cron -f` + the existing `magento-cron` file (or supervisord)	0.5 d	Cron autorestarts, logs rotate, no orphaned PHP processes
S1-4		1 d	

	Tune `my.cnf` (InnoDB buffer pool, log file size, `flush_log_at_trx_commit=2`, `max_connections=400`, slow log)		InnoDB buffer hit rate high; slow-query log actionable
S1-5	Right-size FPM `pm.max_children` from host RAM; set `pm.max_requests=500`	0.5 d	FPM status: `max children reached` stays 0 under load test
S1-6	Build the batched Ofbiz price API + `Collection::afterLoad` pre-fetch; cache in `redis-cache` with TTL	3 d	One ERP round-trip per B2B page instead of N
S1-7	Stagger the colliding `*:15` Ofbiz crons; reconsider per-minute ImageClean	0.25 d	No overlapping ERP calls; fewer cron invocations
S1-8	Pick one reCAPTCHA stack (keep native `Magento_ReCaptcha*`); enable `lock` provider = redis	0.5 d	Single captcha path; lock traffic off MySQL
S1-9	Confirm `pub/health_check.php` ships so the Varnish backend probe passes; verify volume canonicalisation	0.5 d	Varnish reports backend healthy; one media volume in use
S1-10	Decide `Ethnic_UnifiedCache` in prod — enable for edge purge, or document CLI-only edge flush	0.25 d	Admin flush behaviour matches documented expectation
S1-11	Add OpenSearch resilience: connection retry + tight timeouts, alert on `_cluster/health`, evaluate a second node / managed HA (logs show a `NoNodesAvailable` outage)	2 d	Node loss degrades gracefully; alert fires before customer-facing fatals

Phase 2 — Hardening & cleanup QUARTER

Pay down technical debt and add the observability needed to keep the gains.

ID	Action	Effort	Validate
S2-1	Refactor the 22+ `ObjectManager::getInstance()` call sites to constructor DI; rerun `di:compile`	4 d	phpstan ObjectManager warnings → 0; interception intact
S2-2	Stop runtime `env.php` generation; deploy an immutable config + secrets injection	1.5 d	Manual config survives restart; no crypt key in VCS/shell
S2-3	Audit `Typify_TerrariumConfigurator` assets (588 PNGs, 52 MB) — move to CDN, lazy-load	2 d	Module ships only JS/CSS; static-deploy time drops
S2-4	Remove `Typify_CustomFilters` from disk; fix `AdaptiveResize` path allow-list; clean code TODOs/test routes	1 d	Dead plugin gone; no allow-list errors in log
S2-5	Add a log shipper + dashboards (FPM busy children, Redis hit/eviction, Varnish hit rate, InnoDB buffer)	2 d	Single pane of glass; alerting on `evicted_keys` + memory > 85%
S2-6	Add a `.dockerignore`; adopt a multi-stage build for a slim runtime image; add scheduled `mysqldump`	1.5 d	Smaller image; build context lean; restorable backups
S2-7	Benchmark and (if positive) enable tracing JIT; tune Varnish workspace if `ws_overflow` > 0	1 d	5–15% CPU-path gain confirmed before keeping JIT on
S2-8	Harden the `Typify_Shelter` submit observer (`QuoteItemId` lookup); identify and fix the GraphQL caller using the unsupported `visibility` filter / missing `filter` argument	1.5 d	`QuoteItemId not found` warnings stop; GraphQL errors cleared from log

Phase 3 — Strategic ROADMAP

Larger initiatives that change the operating model; plan as their own projects.

ID	Action	Outcome
S3-1	Index Ofbiz B2B prices into Magento's customer-group-price table via a scheduled job	Turns `afterGetPrice` into a read-through fallback only; removes per-request ERP dependency entirely
S3-2	Add RabbitMQ + queue consumers (or formally commit to the DB queue)	Async/bulk operations and scheduled indexers run reliably under load
S3-3	Migrate MySQL auth to `caching_sha2_password`	Forward-compatible with MySQL 8.4 / 9.x
S3-4	Plan the 2.4.9 / 2.4.10 upgrade; re-apply or retire the reverted CSP patch in lockstep	Stays on a supported security line; removes a fragile manual patch
S3-5	Maintain a dedicated `php-spx` "perf" image variant for controlled production profiling	Repeatable profiling without touching the standard runtime image
S3-6	Revisit single-source MSI when a second warehouse appears on the roadmap	Inventory model scales when the business does

Sequencing at a glance

```
Week 1       Phase 0  Stabilise     ▓▓▓▓▓▓▓  remove sharp edges, ship via image rebuild
Weeks 2–4    Phase 1  Resilience    ▓▓▓▓▓▓▓  split Redis, healthchecks, tuning, batched pricing
Quarter      Phase 2  Hardening     ▓▓▓▓▓▓▓  DI cleanup, observability, slim build, backups
Roadmap      Phase 3  Strategic     ▓▓▓▓▓▓▓  ERP price sync, queue, upgrade, MySQL auth
```

Recommended first sprint

Phase 0 in full (≈4 engineering days) plus S1-1 (Redis split) and S1-2 (healthchecks). That combination eliminates both CRITICAL findings and the two most user-visible HIGH risks — worker stalls and random logouts — within the first two weeks, before any larger refactor begins.

Prepared by **The TaaS Company** · Blue Lagoon Magento 2.4.8-p4 production audit · 2 June 2026. This document consolidates the infrastructure & code audit, setup analysis, and Redis analyses into a single production-scoped deliverable. Confidential.